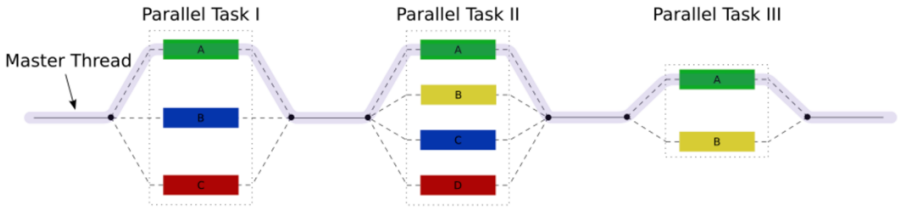
ЛАБОРАТОРИ АЖИЛ №2 ОНОО - 3

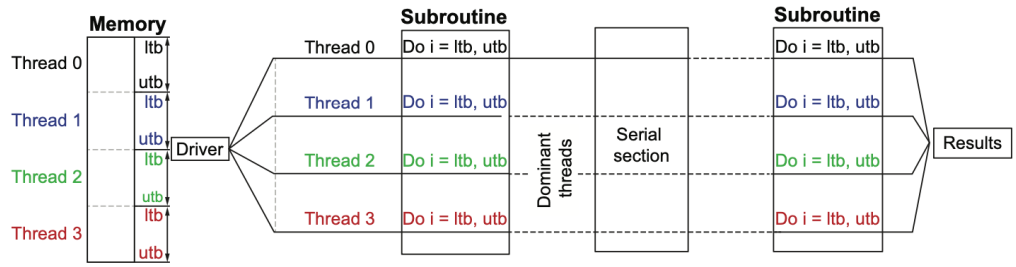
Зорилго: Тус лабораторийн ажлын үндсэн зорилго нь олон цөмт (Multi-core) процессорын тооцооллын нөөцийг оновчтой удирдах, өндөр гүйцэтгэлтэй системийг загварчлан хөгжүүлэхэд оршино. Үүнд дараах даалгаваруудыг хэрэгжүүлнэ.

I ХЭСЭГ

1. **TaskSystemParallelSpawn** (класс) хэрэгжүүлэлт: run() функц ажиллах болгонд үйлдлийн системээс thread үүсгэж, тухайн ажил дуусмагц устгана. Нийт даалгавар num_total_tasks thread-үүдэд хуваарилна. Ажлыг статикаар хувиарлана. Бүх thread ажлыг хийж дууссаны дараа буцаж үндсэн thread-тэй join() хийж нийлүүлнэ үү. (Энэ даалгаврын хувьд ямар сул тал байна вэ?)



2. **TaskSystemParallelThreadPoolSpinning** (класс) хэрэгжүүлэлт: Энэ хэсэгт thread үүсгэх зардлыг арилгахын тулд (**TaskSystemParallelSpawn**) thread pool ашиглана. Өөрөөр хэлбэл програмын эхэнд байгуулагч функц дотор thread-үүдийг ганц удаа үүсгээд, програмыг дуустал устгахгүйгээр дахин ашиглана. Ямар нэг thread – ийн ажил дууссан бол while(true) {} давталтанд ажил ирэхийг хүлээнэ (spinning). Хоёр thread нэг ижил дугаартай ажлыг зэрэг авч хийхгүйн тулд std::mutex ашигла. (Энэ даалгаврын хувьд ямар сул тал байна вэ?)
3. **TaskSystemParallelThreadPoolSleeping** (класс) хэрэгжүүлэлт: CPU-ийн нөөцийг зөвхөн хэрэгтэй үед ашиглаж нөөцийг хэмнэнэ. (Spinning-тэй харьцуулахад) Thread-үүд ажилгүй үедээ түр унтах горимд шилжинэ (wait). Шинэ ажил ирэхэд Мастер thread унтаж буй ажилчдыг сэрээнэ. Бүх ажил дууссаны дараа сүүлчийн ажилчин Мастер thread-ийг сэрээнэ. Үүнийг хэрэгжүүлэхийн тулд std::condition_variable (нөхцөлт хувьсагч)-ийг ашиглаарай.



Програм ажиллаж дуусахад дараах байдлаар үр дүн гарна. Хэрэв хэрэгжүүлэлтийг зөв хийсэн бол Parallel Sleeping Pool нь хугацааны хувьд хамгийн бага хугацаанд ажиллана. Serial > Parallel Spawn > Parallel Spinning Poos > Parallel Sleeping Pool.

```
Testing [Serial System]... Done. Time: 0.0172s
Testing [Parallel Spawn]... Done. Time: 0.0024s
Testing [Parallel Spinning Pool]... Done. Time: 0.0016s
Testing [Parallel Sleeping Pool]... Done. Time: 0.0015s
```

main.cpp: Туршилтыг ажиллуулж, хугацааг хэмжих үндсэн файл.

itasksys.h: Энэ нь бүх Task System-үүдийн суурь интерфейс (Abstract Class) ба энд run() болон sync() гэсэн виртуал функцуудыг тодорхойлсон. Оюутны хэрэгжүүлж буй бүх классууд үүнээс удамшиж ажиллана.

tasksys.h (Header)**:** Лабораторийн заавар дээрх 4 өөр системийг (Serial, Spawn, Spinning, Sleeping) класс болгон энд тодорхойлсон.

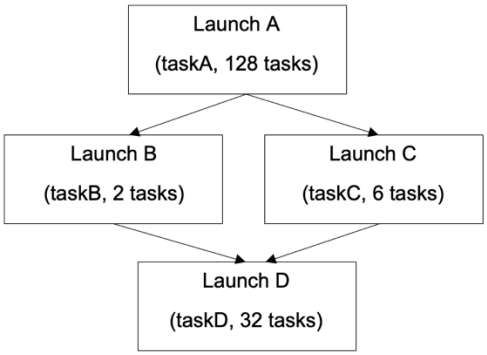
tasksys.cpp (Implementation)**:** Дээрх классуудын логик буюу "биеийг" энд бичнэ. Thread үүсгэх, Mutex ашиглан түгжих, Condition Variable-оор сэрээх зэрэг C++ кодыг энд хэрэгжүүлнэ үү.

II ХЭСЭГ

Зорилго нь ажлуудыг дараалын дагуу, хамаарлыг нь алдагдуулахгүйгээр асинхрон (Background) байдлаар ажиллуулах систем хөгжүүлэхэд оршино.

- runAsyncWithDeps() дуудагдахад систем ажлыг шууд хийж дуусгах албагүй, (I хэсгийн run() -аас ялгаатай.) зөвхөн "дараалалд" бүртгэж аваад шууд буцна. Өөрөөр хэлбэл ажил дуусахыг хүлээхгүйгээр шууд дараагийн мөр рүү шилжих процесс юм. (Non blocking)
- **Хамаарал (Dependencies):** Ажил бүр өөрийн гэсэн TaskID-тэй байна. Шинэ ажил ирэхдээ өмнөх ямар ID-тай ажлууд дуусаны дараа эхлэх ёстойг (dependency list) зааж өгнө.
    - Хэрэв тухайн ажил өөр ажилтай ямар нэгэн хамааралгүй бол систем тухайн ажлыг шууд “ready queue” – д оруулна. Worker урсгалууд боломжтой болмогц дараалын эхнээс ажлыг авч ажиллаж эхэлнэ.
    - Хэрэв ажил өөр ажлаас хамаарч байгаа бол “ready queue” -д орохгүй. Хүлээлгийн жагсаалтанд байна. Өмнөх ажил нь дуусаны дараа систем бэлэн төлөвт шилжүүлнэ.
- **Sync функц:** sync() функц дуудагдах үед л систем тухайн үед дараалалд байсан бүх ажлууд бүрэн дуусахыг хүлээнэ.

Хамаарлыг жишээ зургаар тайлбарлабал



- B болон C ажлууд нь A-аас хамааралтай. Энэ нь A-ийн 128 ажил бүгд дуусахаас нааш B эсвэл C-ийн аль ч ажил эхэлж болохгүй гэсэн үг (Barrier).
- D ажил нь B болон C-ээс хамааралтай. D-ийн 32 ажил эхлэхийн тулд B-ийн 2 ажил, C-ийн 6 ажил бүгд бүрэн дууссан байх ёстой.

Даалгаврыг VI долоо хоногт шалгана.

Эхлэх кодыг https://github.com/AnkhbayarN/F.CSM306-Lab02-Starter авна уу.